Can I make that python unittest executes them following an order?

Ordering Tests in Python `unittest`

Short answer: `unittest` intentionally does **not** guarantee test execution order — tests are sorted alphabetically by default (via `TestLoader.sortTestMethodsUsing`). The framework's design philosophy is that tests should be **independent** and order-agnostic.

However, there are several ways to enforce an order if you really need to.

1. The "proper" way: use `TestSuite` explicitly

Build a suite and add tests in the order you want:

```
python

import unittest

class MyTests(unittest.TestCase):
    def test_a(self): ...
    def test_b(self): ...
    def test_c(self): ...

if __name__ == "__main__":
    loader = unittest.TestLoader()
    suite = unittest.TestSuite()
    # Add in explicit order
    suite.addTest(MyTests("test_c"))
    suite.addTest(MyTests("test_a"))
    suite.addTest(MyTests("test_b"))
    runner = unittest.TextTestRunner()
    runner.run(suite)
```

This works reliably with **any** runner (unittest, pytest, etc.) because the order is baked into the suite.

2. Rename tests to exploit alphabetical sorting

2. Rename tests to exploit alphabetical sorting

Since the default loader sorts by name:

```python
def test_01_setup(self): ...
def test_02_process(self): ...
def test_03_teardown(self): ...
```

Simple, portable, but fragile if you rename things.

3. Custom `TestLoader`

Override `sortTestMethodsUsing`:

```python
class OrderedLoader(unittest.TestLoader):
    def sortTestMethodsUsing(self, a, b):
        # Return -1, 0, 1 based on your desired order
        order = ["test_foo", "test_bar", "test_baz"]
        return order.index(a) - order.index(b)
```

Then run via `OrderedLoader().loadTestsFromTestCase(MyTests)`.

4. Use `pytest-order` (if you can switch runners)

pytest is far friendlier here:

```python
import pytest

@pytest.mark.order(1)
def test_first(): ...

@pytest.mark.order(2)
def test_second(): ...
```

or rely on declaration order (pytest runs tests in source order by default within a module).

⚠️ A word of caution

If tests genuinely depend on order, it's usually a **design smell**:

- Tests can't be run in isolation → harder to debug.
- Parallel execution (e.g., `pytest-xdist`) breaks.
- A failure in one test cascades falsely into others.

Prefer `setUpClass` / `tearDownClass` for shared fixtures, or split dependent logic into a **single test method** that calls helper functions in sequence:

```python
def test_full_workflow(self):
    self.step_create()      # helper, not test_
    self.step_process()
    self.step_verify()
```

Recommendation: Use approach **#1 (explicit `TestSuite`)** if you must control order portably. Otherwise, refactor toward independence — it pays off as the suite grows.

> Can I guarantee from unittest that TestCase classes are executes in a given order

Guaranteeing `TestCase` *class* execution order in `unittest`

Short answer: `unittest` itself provides **no API or guarantee** for ordering *test classes*. The loader discovers modules and classes and there is no supported "class priority" mechanism. Ordering classes requires you to control **how the suite is built** (or which runner you use).

Here's the breakdown of what actually happens and how to enforce it.

What `unittest` actually does

- **Within a class:** methods are sorted alphabetically (or by `sortTestMethodsUsing`).
- **Across classes:** `TestLoader.loadTestsFromModule()` discovers classes **in `dir(module)` order**, which is alphabetical by class name. But this is an **implementation detail**, not a documented guarantee, and it changes depending on:
  - whether you use `loadTestsFromModule`, `discover`, or manual loading,
  - the Python version,
  - how tests are invoked (unittest CLI vs. pytest vs. others).

So relying on class-name alphabetization is fragile.

Reliable approaches

1. Build the suite explicitly (most portable)

Don't use auto-discovery; assemble the suite yourself:

```
python

import unittest

from tests import TestCreate, TestProcess, TestVerify

def ordered_suite():
    suite = unittest.TestSuite()
    suite.addTests(unittest.TestLoader().loadTestsFromTestCase(TestCreate))
    suite.addTests(unittest.TestLoader().loadTestsFromTestCase(TestProcess))
    suite.addTests(unittest.TestLoader().loadTestsFromTestCase(TestVerify))
    return suite

if __name__ == "__main__":
    unittest.TextTestRunner(verbosity=2).run(ordered_suite())
```

This is the **only fully reliable, runner-independent** way. The order is literal in code.

2. `load_tests` protocol (works with discovery + CLI)

If you invoke `python -m unittest discover`, `unittest` looks for a `load_tests(loader, tests, pattern)` function in each package/module. You can use it to impose order:

```python
# tests/__init__.py
import unittest
from . import test_create, test_process, test_verify

def load_tests(loader, standard_tests, pattern):
    suite = unittest.TestSuite()
    for module in (test_create, test_process, test_verify):
        suite.addTests(loader.loadTestsFromModule(module))
    return suite
```

Now `python -m unittest discover -s tests` respects your order. This **is** the documented hook meant for exactly this purpose.

3. Custom `TestLoader`

Subclass and override `loadTestsFromModule` to sort classes by a custom key (e.g., a `priority` attribute you set on each class):

```python
class OrderedLoader(unittest.TestLoader):
    def loadTestsFromModule(self, module, *args, **kwargs):
        tests = super().loadTestsFromModule(module, *args, **kwargs)
        # tests is a _FailedTest-or-TestSuite; unwrap and reorder
        cases = [t for t in tests if isinstance(t, unittest.TestSuite)]
        cases.sort(key=lambda s: getattr(s._tests[0].__class__, "priority", 0))
        return unittest.TestSuite(cases)
```

Then tag classes:

```python
class TestCreate(unittest.TestCase):
```

```
    priority = 1
class TestProcess(unittest.TestCase):
    priority = 2
class TestVerify(unittest.TestCase):
    priority = 3
```

Caveat: you must use this loader everywhere (CLI included, e.g. `python -m unittest` won't pick it up unless you use `unittest.main(testLoader=OrderedLoader())`).

4. Rename classes alphabetically

Prefixing classes (`Test01Create`, `Test02Process`, ...) exploits the alphabetical sort. It **usually** works but is not a documented guarantee — don't build a critical pipeline on it.

5. Switch to pytest (if allowed)

pytest collects test classes **in file/source declaration order** by default, and `pytest-order` gives explicit control:

```
python

@pytest.mark.order(1)
class TestCreate: ...
@pytest.mark.order(2)
class TestProcess: ...
```

More robust than fighting `unittest`'s discovery.

The honest caveat

`unittest`'s docs make no promise about cross-class order because **tests are supposed to be independent of each other**. If `TestVerify` genuinely requires `TestProcess` to have run, you have a shared-state dependency that:

- breaks under `pytest-xdist` / parallel runners,
- breaks under test sharding,

- produces confusing cascading failures,
- makes single-test debugging painful.

Preferred fix: model the dependency with `setUpClass` / `tearDownClass` fixtures, a shared `setUpModule`, or collapse the dependent sequence into one test class with ordered helper methods (`_step_create`, `_step_process`, `_step_verify`) inside a single `test_workflow` method.

Recommendation

Need	Use
One-off ordered run	**Explicit** `TestSuite` (#1)
CLI + discovery, keep order	`load_tests` protocol (#2)
Many classes, reusable scheme	**Custom loader** with `priority` (#3)
Can change runner	**pytest + pytest-order** (#5)
True dependency between classes	**Refactor** — don't order

If you must guarantee class order, **#1 or #2** are the only approaches `unittest` actually sanctions; everything else relies on undocumented behavior.

> Shortest answer, if I build the test suite explicitly, I should execute the run_tests.py file instead of discovery

Yes. If you build the `TestSuite` explicitly in `run_tests.py`, run that file directly:

```
bash

python run_tests.py
```

Not `python -m unittest discover` — discovery ignores your explicit suite and rebuilds its own (alphabetical) one.

Rule of thumb:

- **Explicit suite →** `python run_tests.py`

- **Want discovery** → use the `load_tests` hook in `__init__.py`

Having a directory /tests/central/lots_of_files.py
how can I:
from tests.central import * (means all the classes in every file)
Maybe with the __init__.py?

`from tests.central import *` — pulling in all classes from all modules

`import *` only sees names in the **package's namespace** — i.e., what `tests/central/__init__.py` puts there. So yes, you do it via `__init__.py`, by importing everything you want exposed and (optionally) defining `__all__`.

Option 1 — Dynamic (all files, all public classes)

`tests/central/__init__.py`:

```python
import importlib
import pkgutil
from pathlib import Path

__all__ = []

_pkg_dir = Path(__file__).parent
for _mod_info in pkgutil.iter_modules([str(_pkg_dir)]):
    if _mod_info.name.startswith("_"):
        continue
    _mod = importlib.import_module(f"{__name__}.{_mod_info.name}")
    for _name in dir(_mod):
        if _name.startswith("_"):
            continue
        _obj = getattr(_mod, _name)
        if isinstance(_obj, type):          # keep classes only
            globals()[_name] = _obj
            __all__.append(_name)
```

Then anywhere:

```python
from tests.central import *
```

gets every public class from every non-underscore module in `tests/central/`.

Caveats:

- Name collisions: last module imported wins silently. Add a check if you care.
- Import order = alphabetical (pkgutil default) — deterministic, but classes from later modules override earlier ones.
- Importing the package triggers loading **every** module — heavier, and any module-level error blows up `import tests.central`.

Option 2 — Explicit (recommended if list is stable)

```python
# tests/central/__init__.py
from .lots_of_files import ClassA, ClassB, ClassC
from .other_file import ClassD, ClassE

__all__ = ["ClassA", "ClassB", "ClassC", "ClassD", "ClassE"]
```

Clear, greppable, no surprises. Static analyzers and IDEs actually understand it.

Option 3 — `__getattr__` (lazy, PEP 562)

If `lots_of_files.py` is heavy and you want `import *` to stay cheap:

```python
# tests/central/__init__.py
from . import lots_of_files as _lof

__all__ = [n for n in dir(_lof) if not n.startswith("_") and isinstance(getattr(_lof,
n), type)]

def __getattr__(name):
```

```
def __getattr__(name):
    if name in __all__:
        return getattr(_lof, name)
    raise AttributeError(name)
```

Note: `import *` still resolves every name immediately, so laziness only helps plain `import tests.central` — not the star form.

Important: `import *` at module level only

`from tests.central import *` is a **statement**, only legal at module top level (not inside a function/class). If you meant "get the classes programmatically", that's just:

```
python

import tests.central as central
classes = central.__all__
```

TL;DR

- **Yes, use** `__init__.py`.
- **Dynamic** (Option 1) if "lots of files, always want everything".
- **Explicit** (Option 2) if the set is stable — preferred.
- Watch for **name collisions** and the cost of eagerly importing everything.
- `import *` only works at module scope.

Can I make late imports?

Late / lazy imports in `__init__.py`

Yes. PEP 562 gives you a documented hook: module-level `__getattr__`. Combined with a lazy `__dir__` (and optional `__all__`), you can defer importing submodules until something actually touches a name.

Minimal lazy package

Minimal lazy package

```python
# tests/central/__init__.py

def __getattr__(name):
    # Called only when `name` isn't already defined in this module
    import importlib
    for modname in ("lots_of_files", "other_file"):
        try:
            mod = importlib.import_module(f"{__name__}.{modname}")
        except ImportError:
            continue
        if hasattr(mod, name):
            return getattr(mod, name)
    raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

Now:

```python
from tests.central import ClassA   # only tests.central.lots_of_files is imported, and only when ClassA is accessed
```

Nothing gets imported until the attribute is requested.

Lazy `__all__` + `__dir__` (so `import *` and IDEs work)

```python
# tests/central/__init__.py
import importlib, pkgutil
from pathlib import Path

_MODULES = ("lots_of_files", "other_file")   # or discover via pkgutil

def _find(name):
    for modname in _MODULES:
        mod = importlib.import_module(f"{__name__}.{modname}")
        if hasattr(mod, name):
            return getattr(mod, name)
    raise AttributeError(name)
```

```
def __getattr__(name):
    return _find(name)

def __dir__():
    names = set()
    for modname in _MODULES:
        mod = importlib.import_module(f"{__name__}.{modname}")
        names.update(n for n in vars(mod) if not n.startswith("_"))
    return sorted(names)

def __getattr__(name):  # noqa: F811 (illustrative)
    return _find(name)
```

Important: `from tests.central import *` still **eagerly** resolves every name in `__all__` — so star-import can't be lazy. If laziness matters, avoid `import *` and use explicit imports or attribute access.

Even lazier: don't import sibling modules to *find* the name

The version above imports every candidate module to check for the attribute. If `lots_of_files.py` is heavy and you want to avoid loading it just to answer `hasattr`, register the mapping upfront:

```
python

# tests/central/__init__.py
_REGISTRY = {
    "ClassA": ".lots_of_files",
    "ClassB": ".lots_of_files",
    "ClassD": ".other_file",
}

def __getattr__(name):
    if name in _REGISTRY:
        import importlib
        mod = importlib.import_module(_REGISTRY[name], __name__)
        return getattr(mod, name)
    raise AttributeError(name)

def __dir__():
    return sorted(_REGISTRY)
```

Now only the module that actually owns the requested name is imported. This is the pattern used by big libs (e.g. `scipy`, `pandas` internals).

For your `run_tests.py` case

Because your suite is built explicitly, you don't even need `__init__.py` gymnastics — just import what you need inside `run_tests.py`:

```
python

# run_tests.py
import unittest

def build_suite():
    from tests.central.lots_of_files import TestCreate, TestProcess
    from tests.central.other_file import TestVerify

    suite = unittest.TestSuite()
    loader = unittest.TestLoader()
    for cls in (TestCreate, TestProcess, TestVerify):
        suite.addTests(loader.loadTestsFromTestCase(cls))
    return suite

if __name__ == "__main__":
    unittest.TextTestRunner(verbosity=2).run(build_suite())
```

Imports happen at function-call time = late, and order is explicit. **No `__init__.py` magic required.**

Gotchas

- `from pkg import name` → triggers `__getattr__` ✔
- `import pkg.submod` → normal import machinery, bypasses `__getattr__` ✘
- `hasattr` / `getattr` → triggers `__getattr__` (can be costly if it scans modules)
- `__all__` as a literal list is **eager**; use `__dir__` or a function if you need it dynamic
- Circular imports: lazy `__getattr__` genuinely helps here — module isn't loaded until first access

- Static analyzers/IDEs may not see lazily-exported names; a `TYPE_CHECKING` block fixes that:

```python
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .lots_of_files import ClassA, ClassB
```

TL;DR: Yes — PEP 562 `__getattr__` in `__init__.py`. Prefer a static name→module registry over scanning modules, and remember `import *` defeats laziness. For `run_tests.py`, just put imports inside `build_suite()` and skip the magic entirely.

> Shortest answer, I mean imports after a definition of a TestCase class for example

Yes. In Python, `import` is a normal statement — put it anywhere, including *after* a class definition, inside a function, or inside a method.

```python
class TestBase(unittest.TestCase):
    ...

from tests.central import ClassA   # legal, runs at module load, after the class
```

Or truly late (only when needed):

```python
class TestBase(unittest.TestCase):
    def test_x(self):
        from tests.central import ClassA   # imported on first call
        ...
```

Only rule: `from x import *` is **not** allowed inside a function — module level only.

> Will check_data be executed as a test?:
> class TestDBIntegrity(unittest.TestCase):
> def test_no_env(self):

```
def test_no_env(self):
    db = getdb()
    self.assertTrue(LOG_DIR.exists())
    self.check_data(db)

def test_env_vars(self):
```

No. `check_data` will **not** be executed as a test.

`unittest` only collects methods whose names start with `test`. `check_data` is just a regular helper method you call explicitly from your tests.

But two real problems in your code:

1. `unittest` **test methods receive no arguments beyond** `self`**.** You're calling `self.check_data(db)` — that's fine, it's a plain method call. ✔ (No issue there — just confirming.)
2. `tearDownClass` **will break if** `test_env_vars` **fails mid-way.** If `self.check_data(db)` raises before `del os.environ[...]`, the env var stays set, and if `LOG_DIR` / `DOCKER_VOLUME` / `ENV_LOG_DIR` don't exist (e.g. because `test_no_env` failed first), `shutil.rmtree` raises `FileNotFoundError`. Wrap in `exist_ok=True` or guard:

```python
@classmethod
def tearDownClass(cls) -> None:
    for p in (LOG_DIR, DOCKER_VOLUME, ENV_LOG_DIR):
        if p.exists():
            shutil.rmtree(p)
```

Quick reference for naming rules:

Name	Treated as test?
`test_foo`	✔ yes
`check_data`	✘ no (helper)
`TestFoo` (class)	✔ yes (must subclass `TestCase`)
`_test_foo`	✘ no (leading underscore)

`_test_foo`	✔ no (leading underscore)
`testfoo`	✔ yes (`test` is the only prefix required)

If you ever *do* want it collected, rename to `test_check_data` — but then it needs a `db` fixture in `setUp` , and it would run as an independent test, which you likely don't want here. Keeping it as `check_data` is correct.

> Can I conigure unittest to run my TestCase classes but functions ordered?

Ordered execution of `TestCase` classes *and* their methods in `unittest`

Short answer: **not via configuration** — `unittest` has no config flag for ordering. But you can get **full control over both class order and method order** with a small amount of code. Here are the practical options, shortest first.

1. Simplest: rename methods to sort correctly (methods only)

Default loader sorts methods alphabetically. So:

```python
class TestDBIntegrity(unittest.TestCase):
    def test_01_no_env(self): ...
    def test_02_env_vars(self): ...
```

Class order is still alphabetical by class name → prefix classes too (`Test01_...`). Works, but it's a naming convention, not a guarantee.

2. Explicit suite + custom method order (recommended, no magic)

Order is literal in code — classes **and** methods:

```python
```

```
# run_tests.py
import unittest

def ordered_suite():
    from tests.central.db import TestDBIntegrity
    from tests.central.other import TestOther

    loader = unittest.TestLoader()
    suite = unittest.TestSuite()

    # Classes in the order you list them; methods in the order you list them
    for cls, methods in [
        (TestDBIntegrity, ["test_no_env", "test_env_vars"]),
        (TestOther,       ["test_x", "test_y"]),
    ]:
        for m in methods:
            suite.addTest(cls(m))
    return suite

if __name__ == "__main__":
    unittest.TextTestRunner(verbosity=2).run(ordered_suite())
```

`suite.addTest(cls(method_name))` lets you pick **any method by name in any order**, including non-`test`-prefixed ones if you want.

3. Custom `TestLoader` — one place, applies everywhere

Override both `sortTestMethodsUsing` (method order within a class) and `loadTestsFromModule` (class order):

```
python

import unittest

METHOD_ORDER = {
    "TestDBIntegrity": ["test_no_env", "test_env_vars"],
    "TestOther":       ["test_y", "test_x"],     # reverse, whatever you want
}
CLASS_ORDER = ["TestDBIntegrity", "TestOther"]

class OrderedLoader(unittest.TestLoader):
    def sortTestMethodsUsing(self, a, b):
        cls_order = self._current_class_order or []
```

```
        try:
            return cls_order.index(a) - cls_order.index(b)
        except ValueError:
            return (a > b) - (a < b)          # fallback: alphabetical

    def loadTestsFromTestCase(self, cls):
        self._current_class_order = METHOD_ORDER.get(cls.__name__)
        try:
            return super().loadTestsFromTestCase(cls)
        finally:
            self._current_class_order = None

    def loadTestsFromModule(self, module, *args, **kwargs):
        suite = super().loadTestsFromModule(module, *args, **kwargs)
        cases = list(suite)
        cases.sort(key=lambda s: CLASS_ORDER.index(s._tests[0].__class__.__name__)
                                 if s._tests and s._tests[0].__class__.__name__ in CLASS_
ORDER
                                 else 999)
        return unittest.TestSuite(cases)
```

Use it as `unittest.main(testLoader=OrderedLoader())` or `OrderedLoader().discover("tests")`. The `sortTestMethodsUsing` override is the documented hook for **method** order; class order via `loadTestsFromModule` is the only place you can reorder classes portably.

4. `load_tests` hook — works with `python -m unittest discover`

If you use discovery, define `load_tests` in the package/module and assemble the ordered suite there:

```
python

# tests/central/__init__.py
import unittest
from .db import TestDBIntegrity
from .other import TestOther

def load_tests(loader, standard_tests, pattern):
    suite = unittest.TestSuite()
    for cls, methods in [
```

```
        (TestDBIntegrity, ["test_no_env", "test_env_vars"]),
        (TestOther,       ["test_x", "test_y"]),
    ]:
        for m in methods:
            suite.addTest(cls(m))
    return suite
```

Then `python -m unittest discover -s tests` respects your order — no custom runner needed. This **is** the documented hook for exactly this purpose.

5. If you can switch runners: `pytest`

Declaration order is respected by default for both classes and functions; `pytest-order` gives explicit control:

```
python

@pytest.mark.order(1)
class TestDBIntegrity:
    @pytest.mark.order(1)
    def test_no_env(self): ...

    @pytest.mark.order(2)
    def test_env_vars(self): ...
```

Summary table

Goal	Mechanism	Guaranteed?
Method order only	`sortTestMethodsUsing`	✔ documented hook
Class order only	`loadTestsFromModule` / explicit suite	✔ (custom code)
Both, one file	**Explicit** `TestSuite` (#2)	✔ fully
Both, with `discover`	`load_tests` (#4)	✔ documented hook
Convention only	`test_01_…` / `Test01…`	✘ implicit sort

Configuration flag	—	**doesn't exist**

⚠️ Worth repeating

If `test_env_vars` **must** run after `test_no_env`, that's a shared-state dependency. It breaks under parallel runners (`pytest-xdist`, sharding), makes single-test debugging misleading, and cascading failures get confusing. Preferred refactor: put the dependency in `setUpClass` / `setUp`, or collapse into a single `test_workflow` method calling private helpers (`_step_no_env`, `_step_env_vars`) in sequence — then no ordering config is needed at all.

Shortest answer remove file

`Path.unlink()` (Python 3.x):

```python
from pathlib import Path
Path("file.txt").unlink()
```

Add `missing_ok=True` to ignore if absent:

```python
Path("file.txt").unlink(missing_ok=True)
```

Old style: `os.remove("file.txt")`

what is sys.pycache_prefix dir does not exist

`sys.pycache_prefix` tells Python where to write `.pyc` bytecode cache files instead of the default `__pycache__` folders next to the source.

```python
import sys
sys.pycache_prefix = "/tmp/pycache"
```

If **that directory does not exist**, Python silently **does not write** the `.pyc` files — it does **not** create the directory for you, and it does **not** raise an error. Bytecode caching is simply skipped (source is recompiled each run). Note: since Python 3.8 you *can* set it via `PYTHONPYCACHEPREFIX` env var too, same behavior.

Fix: create it yourself before/at startup:

```
python

import os, sys
prefix = "/tmp/pycache"
os.makedirs(prefix, exist_ok=True)
sys.pycache_prefix = prefix
```

Key points:

- Not auto-created → no error, just no caching.
- Must exist **before** modules are imported (set it as early as possible, e.g. `sitecustomize.py` or top of entry script).
- The env var `PYTHONPYCACHEPREFIX` has the same "must exist" rule.

pycache_dir = Path() ok?

No. `Path()` (no argument) means **the current directory** (`.`), not a pycache dir.

```
python

Path()        # == Path('.')
Path("")      # also == Path('.')
```

So `sys.pycache_prefix = Path()` would write caches relative to CWD — almost certainly not what you want, and it exists by definition so caching "works" but pollutes your working dir.

What you probably want:

```
python

import os, sys
from pathlib import Path

pycache_dir = Path("/tmp/pycache")   # or wherever
```

```
pycache_dir.mkdir(parents=True, exist_ok=True)
sys.pycache_prefix = str(pycache_dir)   # NOTE: must be str, not Path
```

Important: `sys.pycache_prefix` expects a `str`, not a `Path`. Assigning a `Path` object raises `TypeError` (it's a C-level string field). So:

```
python

sys.pycache_prefix = str(pycache_dir)   # ✔
sys.pycache_prefix = pycache_dir        # ✘ TypeError
```

And the dir must exist **before** imports happen — set it at the very top of your entry script (or via `PYTHONPYCACHEPREFIX`), not after `import foo`.

Something is wrong:
#!/bin/env python3

import unittest
import sys
from pathlib import Path
from tests import central

What's wrong

The `pycache_prefix` is set too late. By the time your script reaches `sys.pycache_prefix = str(pycdir)`, `tests.central` (and everything it imports) has **already been imported** at the top of the file — so all `.pyc` files were written/read using the *default* location. Setting the prefix afterwards has no effect on already-imported modules.

`import` runs before the `if __name__ == "__main__"` block. So the order is:

```
python

from tests import central          # ← imports happen HERE, pycache_prefix not set ye
t
...
if __name__ == "__main__":
    sys.pycache_prefix = str(pycdir)   # ← too late for tests.central
```

Fix: set the prefix before importing your test modules

```python
#!/bin/env python3

import sys
from pathlib import Path

# 1) configure pycache FIRST
pycdir = Path(__file__).parent / ".log/pycache"
pycdir.mkdir(0o700, parents=True, exist_ok=True)
sys.pycache_prefix = str(pycdir)

# 2) only now import things that should be cached there
import unittest
from tests import central

if __name__ == "__main__":
    loader = unittest.TestLoader()
    suite = unittest.TestSuite()
    suite.addTests(loader.loadTestsFromTestCase(central.TestParams))
    suite.addTests(loader.loadTestsFromTestCase(central.TestDBIntegrity))
    unittest.TextTestRunner(verbosity=2, buffer=True).run(suite)
```

Note: `import unittest` and `import sys` themselves still get cached in the default place — the stdlib is already compiled and frozen, so that's fine. What matters is that `tests.*` **is imported after** the prefix is set.

Other things worth flagging

1. `mkdir(0o700, parents=True, exist_ok=True)` — args are correct

`Path.mkdir(mode=0o777, parents=False, exist_ok=False)` — your positional `0o700` maps to `mode`. ✔ Fine. (Some people get bitten because `mode` is affected by umask, so the actual perms may be `0o700 & ~umask`. If that matters, `os.chmod` after.)

2. Shebang is wrong

`#!/bin/env python3` → should be `#!/usr/bin/env python3` (with `usr`). As written, it only works if there's literally a `/bin/env`, which most systems don't have.

3. `Path(__file__).parent` is fine, but be aware of CWD

`__file__` is relative to CWD when the script is invoked with a relative path. Using `.resolve()` is safer:

```python
pycdir = Path(__file__).resolve().parent / ".log/pycache"
```

Not the bug here, but avoids surprises when run from another directory.

4. `pycache_dir` vs. what tests do

If your tests **delete** `.log/` in `tearDownClass` (as in your earlier `shutil.rmtree(LOG_DIR)`), and `pycdir` lives under `.log/pycache`, you'll wipe the bytecode cache mid-run. Modules already imported in memory are unaffected, but **later-imported** modules (e.g. late imports inside test methods) will recompile — or worse, if you `rmtree` while a module is being written, you can get corrupted `.pyc` files. Keep the cache out of directories the tests clean up:

```python
pycdir = Path("/tmp/pytest-pycache")   # or Path.home() / ".cache/..."
```

5. `loadTestsFromTestCase` is fine, but...

Order of classes is exactly the order of your two `addTests` calls — ✔ explicit, portable. Method order within each class is alphabetical — if you need explicit method order, use the `OrderedLoader` / explicit `cls("method_name")` approach from earlier.

6. `buffer=True` + `setUpClass` side effects

`buffer=True` captures stdout/stderr per test; output from `setUpClass` / `tearDownClass` is **not** captured the same way (it's attached to the *class* result). Minor, just so you're not surprised that class-level prints still show up.

TL;DR

- **Real bug:** `sys.pycache_prefix` must be set **before** `from tests import central`. Reorder the imports.
- **Cosmetic fixes:** shebang `#!/usr/bin/env python3`, `.resolve()` for `__file__`.
- **Watch out:** don't put the cache inside a dir your `tearDownClass` deletes.
- `sys.pycache_prefix` only affects **new** imports, never already-loaded modules — that's the whole reason ordering matters.